

# **Отчёт по лабораторной работе №12**

**по курсу: Архитектура компьютера и операционные системы**

Иванова Анастасия Сергеевна

# Содержание

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                                            | <b>4</b>  |
| <b>2</b> | <b>Выполнение лабораторной работы</b>                         | <b>5</b>  |
| 2.1      | Задание 1. Резервная копия самого скрипта . . . . .           | 5         |
| 2.2      | Задание 2. Обработка произвольного числа аргументов . . . . . | 8         |
| 2.3      | Задание 3. Аналог команды ls . . . . .                        | 10        |
| 2.4      | Задание 4. Подсчёт файлов по расширению . . . . .             | 14        |
| <b>3</b> | <b>Контрольные вопросы</b>                                    | <b>17</b> |
| <b>4</b> | <b>Вывод</b>                                                  | <b>20</b> |

## Список иллюстраций

|      |                                             |    |
|------|---------------------------------------------|----|
| 2.1  | Создание директории . . . . .               | 5  |
| 2.2  | Создание файла скрипта . . . . .            | 6  |
| 2.3  | Код скрипта backup_self.sh . . . . .        | 6  |
| 2.4  | Изменение прав доступа . . . . .            | 7  |
| 2.5  | Запуск скрипта . . . . .                    | 7  |
| 2.6  | Проверка создания резервной копии . . . . . | 8  |
| 2.7  | Создание файла скрипта . . . . .            | 8  |
| 2.8  | Код скрипта print_args.sh . . . . .         | 9  |
| 2.9  | Запуск скрипта с 8 аргументами . . . . .    | 10 |
| 2.10 | Создание файла скрипта . . . . .            | 10 |
| 2.11 | Код скрипта myls.sh . . . . .               | 11 |
| 2.12 | Запуск скрипта myls.sh . . . . .            | 13 |
| 2.13 | Запуск myls.sh для /etc . . . . .           | 13 |
| 2.14 | Создание файла скрипта . . . . .            | 14 |
| 2.15 | Код скрипта count_files.sh . . . . .        | 15 |
| 2.16 | Создание тестовых файлов . . . . .          | 16 |
| 2.17 | Запуск скрипта . . . . .                    | 16 |
| 2.18 | Подсчёт файлов с расширением jpg . . . . .  | 16 |

# 1 Цель работы

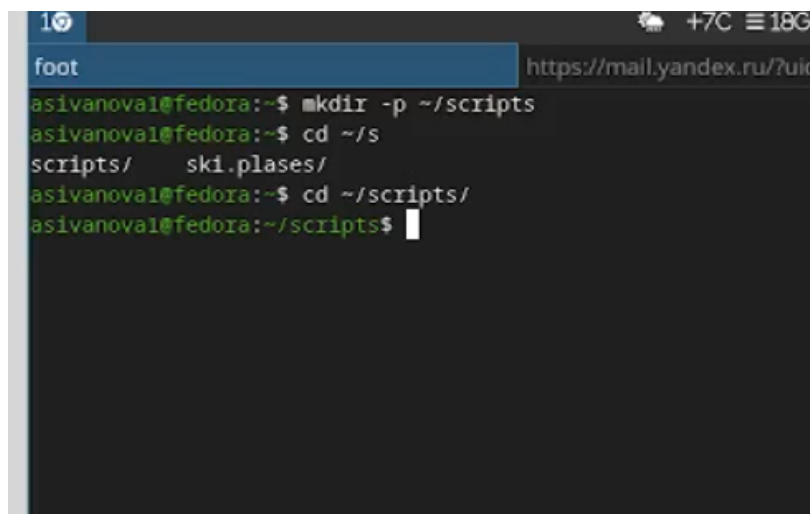
Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

## 2 Выполнение лабораторной работы

### 2.1 Задание 1. Резервная копия самого скрипта

Создадим скрипт, который при запуске делает резервную копию самого себя в директорию ~/backup с помощью архиватора tar.

1. Создадим директорию для скриптов и перейдём в неё (рис. 2.1):

A screenshot of a terminal window with a dark background. The prompt is 'asivanova1@fedora:~\$'. The user enters 'mkdir -p ~/scripts', followed by 'cd ~/s', then 'scripts/' (which is corrected to 'scripts/'), and finally 'cd ~/scripts/'. The prompt changes to 'asivanova1@fedora:~/scripts\$'.

```
asivanova1@fedora:~$ mkdir -p ~/scripts
asivanova1@fedora:~$ cd ~/s
scripts/      ski.places/
asivanova1@fedora:~$ cd ~/scripts/
asivanova1@fedora:~/scripts$
```

Рисунок 2.1: Создание директории

2. Создадим файл скрипта backup\_self.sh (рис. 2.2):

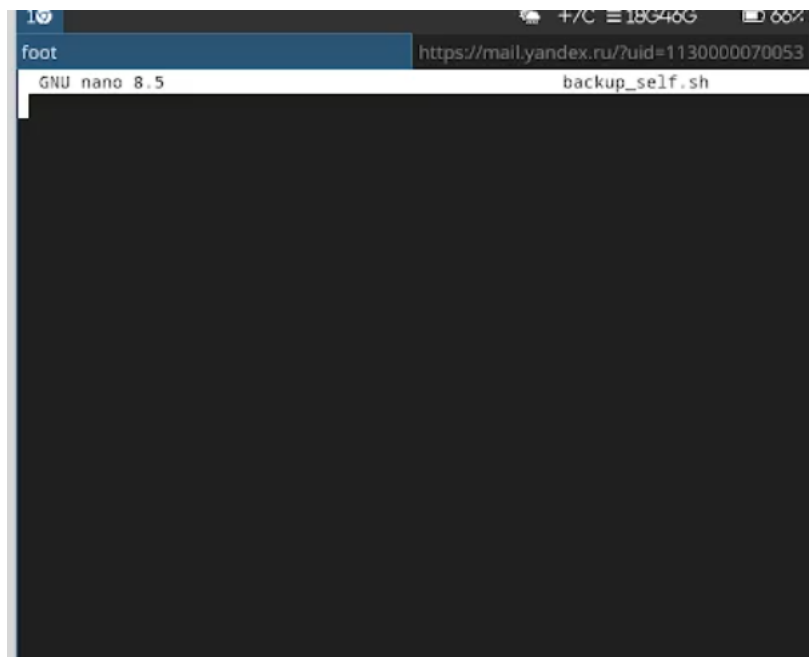


Рисунок 2.2: Создание файла скрипта

3. Код скрипта (рис. 2.3):

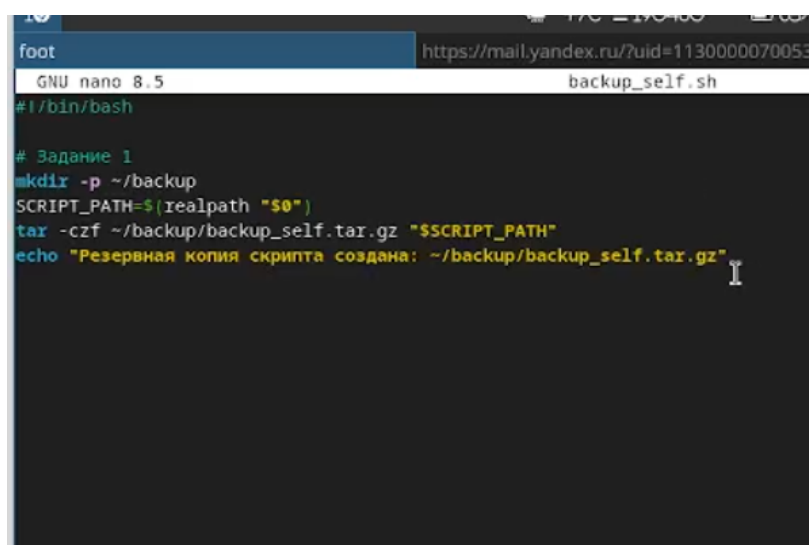
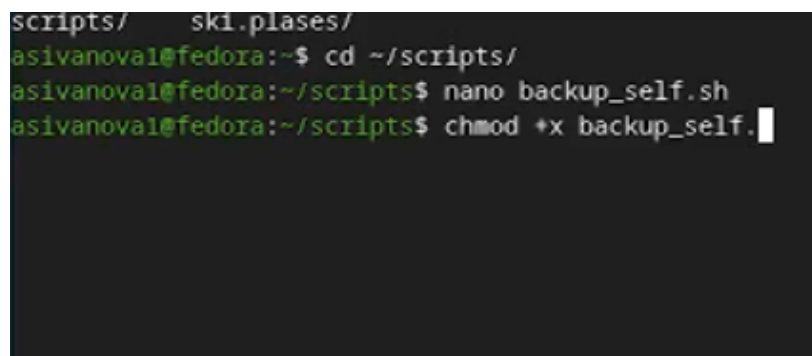


Рисунок 2.3: Код скрипта backup\_self.sh

```
#!/bin/bash
# Задание 1: Резервная копия самого скрипта
mkdir -p ~/backup
SCRIPT_PATH=$(realpath "$0")
tar -czf ~/backup/backup_self.tar.gz "$SCRIPT_PATH"
echo "Резервная копия скрипта создана: ~/backup/backup_self.tar.gz"
```

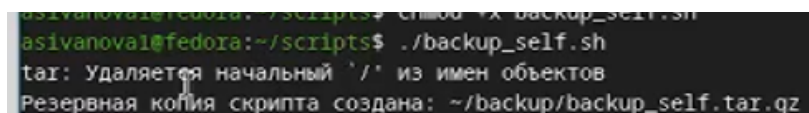
4. Сделаем скрипт исполняемым (рис. 2.4):



```
scripts/      ski.plases/
asivanova@fedora:~$ cd ~/scripts/
asivanova@fedora:~/scripts$ nano backup_self.sh
asivanova@fedora:~/scripts$ chmod +x backup_self.sh
```

Рисунок 2.4: Изменение прав доступа

5. Запустим скрипт (рис. 2.5):



```
asivanova@fedora:~/scripts$ chmod +x backup_self.sh
asivanova@fedora:~/scripts$ ./backup_self.sh
tar: Удаляется начальный '/' из имен объектов
Резервная копия скрипта создана: ~/backup/backup_self.tar.gz
```

Рисунок 2.5: Запуск скрипта

6. Проверим результат (рис. 2.6):

```

Резервная копия скрипта создана: ~/backup/backup_self.tar.gz
asivanova1@fedora:~/scripts$ ls -la ~/backup/
итого 4
drwxr-xr-x. 1 asivanova1 asivanova1   36 апр 26 18:54 .
drwx----- 1 asivanova1 asivanova1 1780 апр 26 18:54 ..
-rw-r--r-- 1 asivanova1 asivanova1   315 апр 26 18:54 backup_self.tar.gz
asivanova1@fedora:~/scripts$ tar -tzf ~/backup/backup_self.tar.gz
home/asivanova1/scripts/backup_self.sh
asivanova1@fedora:~/scripts$

```

Рисунок 2.6: Проверка создания резервной копии

## 2.2 Задание 2. Обработка произвольного числа аргументов

Создадим скрипт, обрабатывающий любое произвольное число аргументов командной строки.

1. Создадим файл скрипта `print_args.sh` (рис. 2.7):

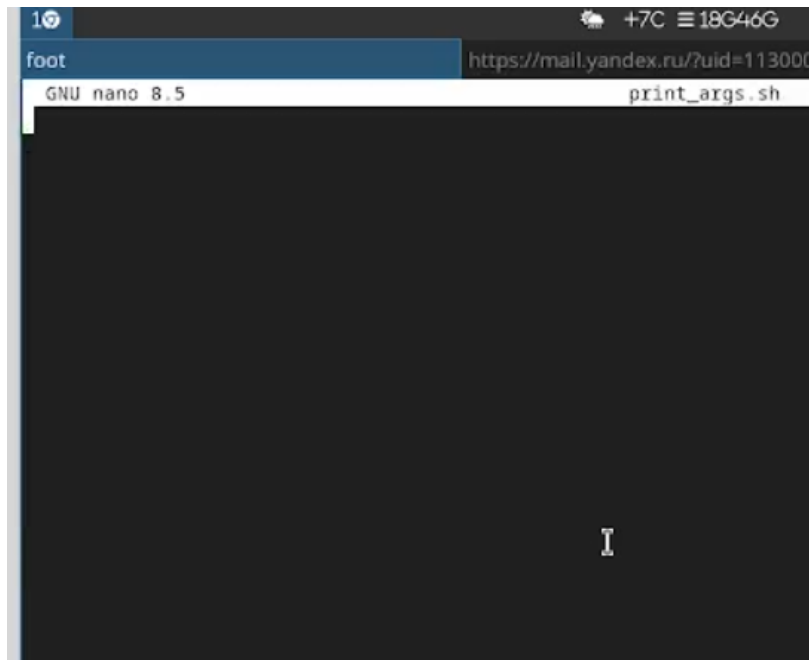
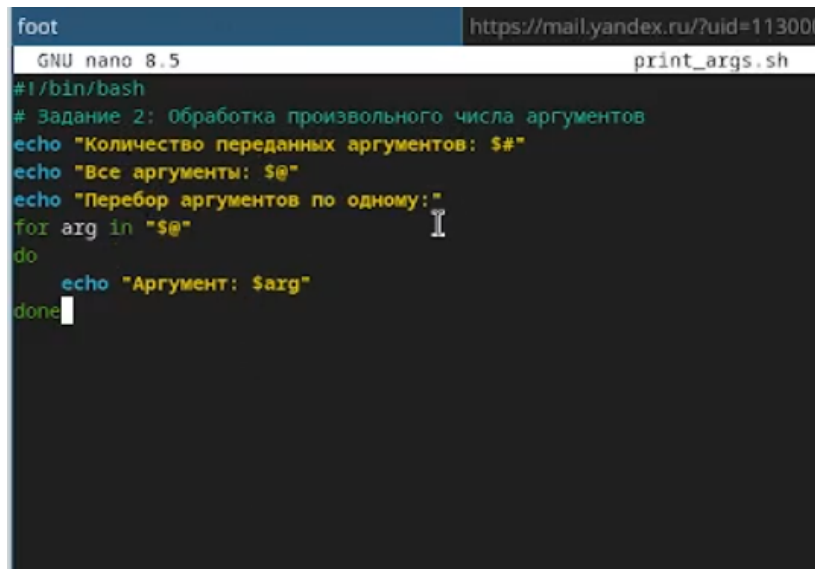


Рисунок 2.7: Создание файла скрипта



2. Код скрипта (рис. 2.8):



```
foot https://mail.yandex.ru/?uid=113000
GNU nano 8.5 print_args.sh
#!/bin/bash
# Задание 2: обработка произвольного числа аргументов
echo "Количество переданных аргументов: $#"
```

Рисунок 2.8: Код скрипта print\_args.sh

```
#!/bin/bash
# Задание 2: Обработка произвольного числа аргументов
echo "Количество переданных аргументов: $#"
```

3. Сделаем скрипт исполняемым и запустим с 8 аргументами (рис. 2.9):

```
asivanova1@fedora:~/scripts$ chmod +x print_args.sh
asivanova1@fedora:~/scripts$ ./print_args.sh a b c d e f g h
Количество переданных аргументов: 8
Все аргументы: a b c d e f g h
Перебор аргументов по одному:
Аргумент: a
Аргумент: b
Аргумент: c
Аргумент: d
Аргумент: e
Аргумент: f
Аргумент: g
Аргумент: h
asivanova1@fedora:~/scripts$
```

Рисунок 2.9: Запуск скрипта с 8 аргументами

## 2.3 Задание 3. Аналог команды ls

Создадим скрипт — аналог команды `ls` (без использования самой команды и `dir`).

1. Создадим файл скрипта `mysls.sh` (рис. 2.10):

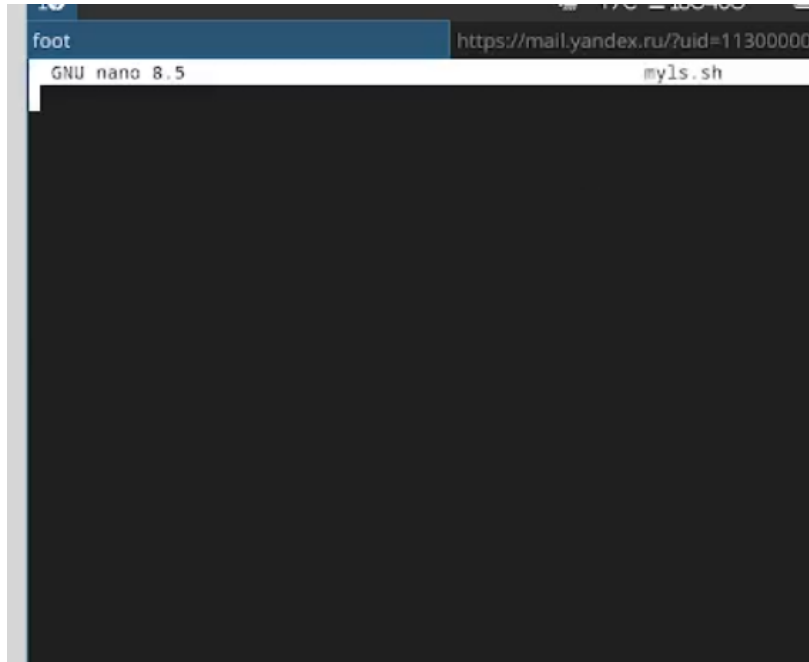
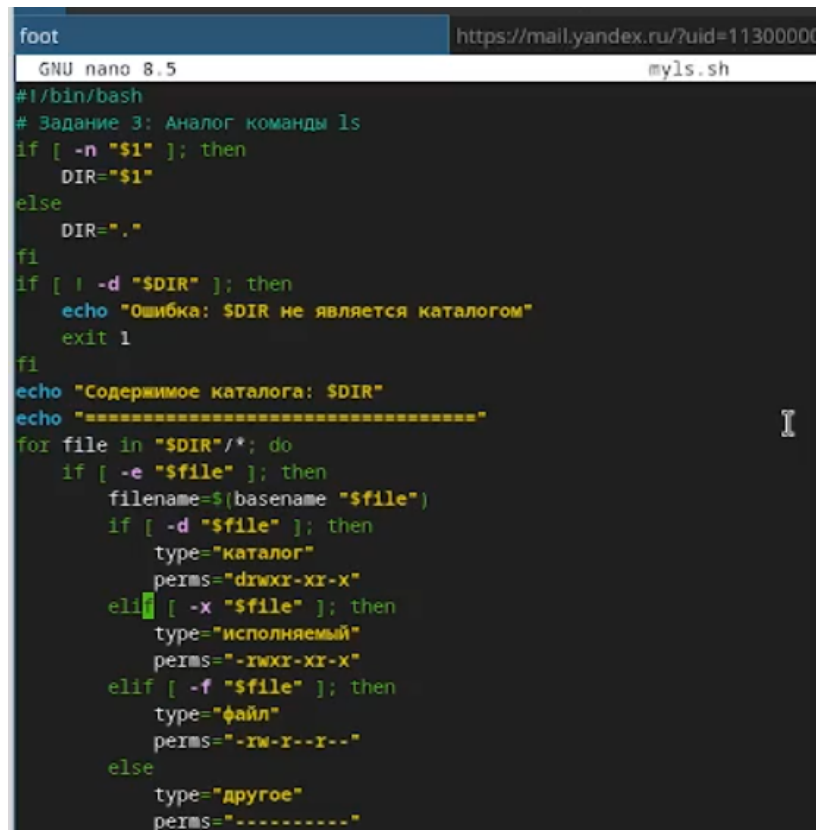


Рисунок 2.10: Создание файла скрипта

2. Код скрипта (рис. 2.11):



```
#!/bin/bash
# Задание 3: Аналог команды ls
if [ -n "$1" ]; then
    DIR="$1"
else
    DIR="."
fi
if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом"
    exit 1
fi
echo "Содержимое каталога: $DIR"
echo "=====I"
for file in "$DIR"/*; do
    if [ -e "$file" ]; then
        filename=${basename "$file"}
        if [ -d "$file" ]; then
            type="каталог"
            perms="drwxr-xr-x"
        elif [ -x "$file" ]; then
            type="исполняемый"
            perms="-rwxr-xr-x"
        elif [ -f "$file" ]; then
            type="файл"
            perms="-rw-r--r--"
        else
            type="другое"
            perms="-----"
        fi
    fi
done
```

Рисунок 2.11: Код скрипта myls.sh

```
#!/bin/bash
# Задание 3: Аналог команды ls
if [ -n "$1" ]; then
    DIR="$1"
else
    DIR="."
fi
if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом"
    exit 1
fi
```

```

fi
echo "Содержимое каталога: $DIR"
echo "====="
for file in "$DIR"/*; do
    if [ -e "$file" ]; then
        filename=$(basename "$file")

        if [ -d "$file" ]; then
            type="каталог"
            perms="drwxr-xr-x"
        elif [ -x "$file" ]; then
            type="исполняемый"
            perms="-rwxr-xr-x"
        elif [ -f "$file" ]; then
            type="файл"
            perms="-rw-r--r--"
        else
            type="другое"
            perms="-----"
        fi
        echo "$perms $filename ($type)"
    fi
done

```

3. Запустим скрипт (рис. 2.12):

```
asivanova@fedora:~/scripts$ chmod +x myls.sh
asivanova@fedora:~/scripts$ ./mysls.sh
Содержимое каталога: .
=====
-rwxr-xr-x backup_self.sh (исполняемый)
-rwxr-xr-x myls.sh (исполняемый)
-rwxr-xr-x print_args.sh (исполняемый)
asivanova@fedora:~/scripts$
```

Рисунок 2.12: Запуск скрипта myls.sh

4. Запустим скрипт для каталога /etc (рис. 2.13):

```
-rwxr-xr-x print_args.sh (исполняемый)
asivanova@fedora:~/scripts$ ./mysls.sh /etc
Содержимое каталога: /etc
=====
drwxr-xr-x abrt (каталог)
-rw-r--r-- adjtime (файл)
-rw-r--r-- aliases (файл)
drwxr-xr-x alsa (каталог)
drwxr-xr-x alternatives (каталог)
drwxr-xr-x anaconda (каталог)
-rw-r--r-- anacrontab (файл)
-rw-r--r-- asound.conf (файл)
-rw-r--r-- at.deny (файл)
drwxr-xr-x audit (каталог)
drwxr-xr-x authselect (каталог)
drwxr-xr-x avahi (каталог)
drwxr-xr-x bash_completion.d (каталог)
-rw-r--r-- bashrc (файл)
-rw-r--r-- bindresvport.blacklist (файл)
drwxr-xr-x binfmt.d (каталог)
drwxr-xr-x bluetooth (каталог)
-rw-r--r-- chrony.conf (файл)
drwxr-xr-x cifs-utils (каталог)
drwxr-xr-x cockpit (каталог)
drwxr-xr-x credstore (каталог)
drwxr-xr-x credstore.encrypted (каталог)
drwxr-xr-x cron.d (каталог)
drwxr-xr-x cron.daily (каталог)
```

Рисунок 2.13: Запуск myls.sh для /etc

## 2.4 Задание 4. Подсчёт файлов по расширению

Создадим скрипт, подсчитывающий количество файлов с заданным расширением в указанной директории.

1. Создадим файл скрипта `count_files.sh` (рис. 2.14):

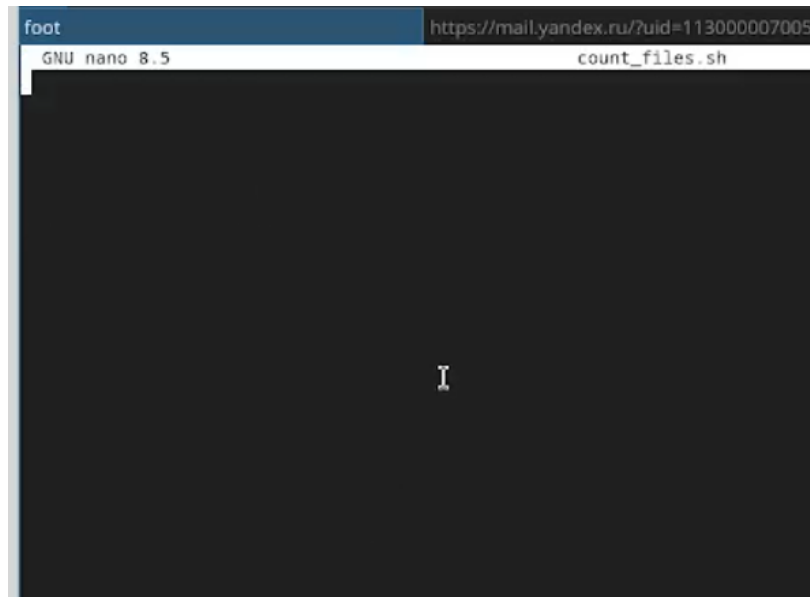
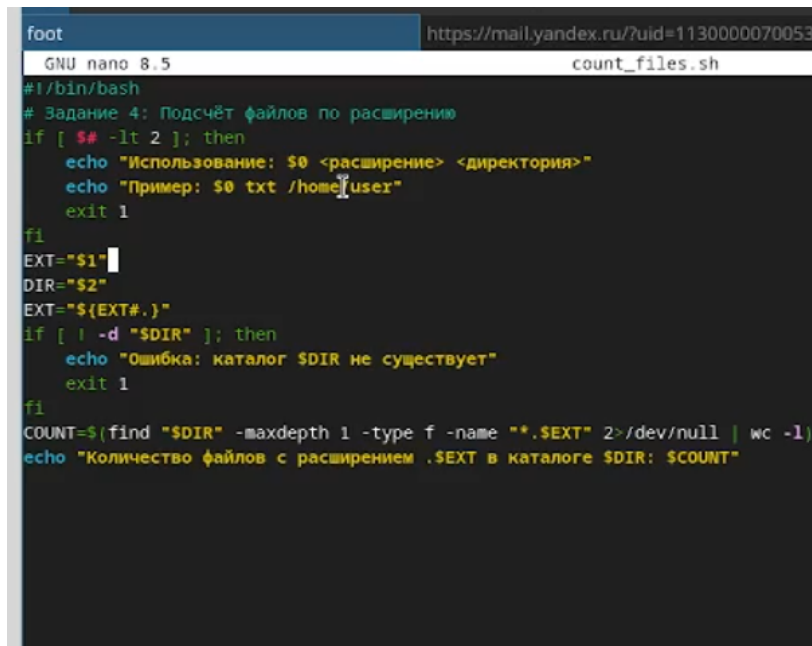


Рисунок 2.14: Создание файла скрипта

2. Код скрипта (рис. 2.15):



```
foot https://mail.yandex.ru/?uid=1130000070053
GNU nano 8.5 count_files.sh
#!/bin/bash
# Задание 4: Подсчёт файлов по расширению
if [ $# -lt 2 ]; then
    echo "Использование: $0 <расширение> <директория>"
    echo "Пример: $0 txt /home/user"
    exit 1
fi
EXT="$1"
DIR="$2"
EXT="${EXT#}"
if [ ! -d "$DIR" ]; then
    echo "Ошибка: каталог $DIR не существует"
    exit 1
fi
COUNT=$(find "$DIR" -maxdepth 1 -type f -name "*.EXT" 2>/dev/null | wc -l)
echo "Количество файлов с расширением .EXT в каталоге $DIR: $COUNT"
```

Рисунок 2.15: Код скрипта count\_files.sh

```
#!/bin/bash
# Задание 4: Подсчёт файлов по расширению
if [ $# -lt 2 ]; then
    echo "Использование: $0 <расширение> <директория>"
    echo "Пример: $0 txt /home/user"
    exit 1
fi
EXT="$1"
DIR="$2"
EXT="${EXT#}"
if [ ! -d "$DIR" ]; then
    echo "Ошибка: каталог $DIR не существует"
    exit 1
fi
COUNT=$(find "$DIR" -maxdepth 1 -type f -name "*.EXT" 2>/dev/null | wc -l)
```

```
echo "Количество файлов с расширением .$EXT в каталоге $DIR: $COUNT"
```

3. Создадим тестовые файлы (рис. 2.16):

```
asivanova1@fedora:~$ cd ~/scripts/
asivanova1@fedora:~/scripts$ touch ~/test1.txt ~/test2.txt ~/test3.txt
asivanova1@fedora:~/scripts$ mkdir -p ~/testdir
asivanova1@fedora:~/scripts$ touch ~/testdir/file1.txt ~/testdir/file2.jpg
asivanova1@fedora:~/scripts$ ls
backup_self.sh  count_files.sh  myls.sh  print_args.sh
asivanova1@fedora:~/scripts$ cd
asivanova1@fedora:~$ ls
1.cpp      Documents      'elab07.sh'  scripts      test_мс      Изображения
abcl       Downloads      lab07.sh     ski_places   work         Музыка
australia  feathurs      LICENSE     test1.txt    work_lsem    Общедоступные
backup     file.txt      my_os       test2.txt    видео       'Рабочий стол'
conf.txt   git-extended  play        test3.txt    документы   Шаблоны
Desktop    gitHubDesktop-linux-amd64-3.4.7-linux1.deb  play_catalog testdir      Загрузки
```

Рисунок 2.16: Создание тестовых файлов

4. Запустим скрипт (рис. 2.17):

```
asivanova1@fedora:~/scripts$ ./count_files.sh txt ~/
Количество файлов с расширением .txt в каталоге /home/asivanova1/: 2
asivanova1@fedora:~/scripts$ ./count_files.sh txt ~/scripts/
Количество файлов с расширением .txt в каталоге /home/asivanova1/scripts/: 3
asivanova1@fedora:~/scripts$
```

Рисунок 2.17: Запуск скрипта

5. Запустим скрипт для другого расширения (рис. 2.18):

```
Количество файлов с расширением .txt в каталоге /home/asivanova1/scripts/: 3
asivanova1@fedora:~/scripts$ ./count_files.sh jpg ~/scripts/testdir/
Количество файлов с расширением .jpg в каталоге /home/asivanova1/scripts/testdir/: 1
asivanova1@fedora:~/scripts$
```

Рисунок 2.18: Подсчёт файлов с расширением jpg



## 3 Контрольные вопросы

### 1. Что такое командная оболочка? Приведите примеры.

Командная оболочка (shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой посредством команд. Примеры: sh, bash, csh, ksh, zsh.

### 2. Что такое POSIX?

POSIX (Portable Operating System Interface) — набор стандартов, описывающих интерфейсы между операционной системой и прикладными программами, обеспечивающий совместимость различных UNIX-подобных систем.

### 3. Как определяются переменные и массивы в языке программирования bash?

Переменные определяются присваиванием: var=value. Массивы создаются командой set -A arr val1 val2 или arr=(val1 val2). Обращение к элементу массива: \${arr[0]}.

### 4. Каково назначение операторов let и read?

- let — выполняет арифметические вычисления
- read — считывает данные со стандартного ввода (клавиатуры)

### 5. Какие арифметические операции можно применять?

Сложение (+), вычитание (-), умножение (\*), деление (/), остаток от деления (%), возведение в степень (\*\*), а также побитовые операции.

### 6. Что означает операция (( ))?

Двойные круглые скобки используются для выполнения арифметических вычислений и условных выражений. Например: `(( sum = x + y ))`.

### 7. Какие стандартные имена переменных Вам известны?

`PATH, HOME, USER, SHELL, PWD, IFS, MAIL, TERM, LOGNAME.`

### 8. Что такое метасимволы?

Метасимволы — символы, имеющие специальное значение: `*`, `?`, `[`, `]`, `>`, `<`, `|`, `&`, `;`, `$`, `\`, ```.

### 9. Как экранировать метасимволы?

- `\` — экранирование одного символа
- `'...'` — полное экранирование (все метасимволы теряют смысл)
- `"..."` — частичное экранирование (кроме `$`, ```, `\`)

### 10. Как создавать и запускать командные файлы?

Создать файл с первой строкой `#!/bin/bash`, сделать его исполняемым (`chmod +x file.sh`), запустить: `./file.sh` или `bash file.sh`.

### 11. Как определяются функции в языке программирования bash?

```
function name { commands; }  
# или  
name() { commands; }
```

### 12. Каким образом можно выяснить, является файл каталогом или обычным файлом?

Используя условные операторы: `[ -d "$file" ]` — каталог, `[ -f "$file" ]` — обычный файл.

### 13. Каково назначение команд `set`, `typeset` и `unset`?

- `set` — установка параметров оболочки
- `typeset` — задание атрибутов переменных
- `unset` — удаление переменных или функций

**14. Как передаются параметры в командные файлы?**

Через позиционные параметры: \$1, \$2, ... \$9, \${10} для 10 и более.

**15. Назовите специальные переменные языка bash и их назначение.**

- \$0 — имя скрипта
- \$1-\$9 — параметры
- \$# — количество параметров
- \$@ — все параметры
- \$? — код возврата
- \$\$ — PID процесса
- \$! — PID последнего фонового процесса

## 4 Вывод

Мы изучили основы программирования в оболочке `bash`, научились создавать командные файлы, работать с переменными и массивами, использовать управляющие конструкции `for`, `if`, организовывать поиск файлов и обрабатывать аргументы командной строки. В ходе работы были разработаны четыре скрипта: для создания резервной копии, обработки произвольного числа аргументов, аналог команды `ls` и подсчёт файлов по расширению.